Proyecto Final Backend · Soluciones para el CLENA

Documento de arquitectura

Guías CLENA: página web de autoaprendizaje para la sala de internet

Dato	Detalle
Proyecto	Guías CLENA (Autoaprendizaje digital)
Entrega	Entrega 1 · Entregable E5
Grupo	Grupo 4
Integrantes	Ivan Parra, Marcus Rambal, Juan Romero, Daniel Cruzado
Repositorio	github.com/MarcusRambal/Autoaprendizaje-digital---Clena
Fecha	13 de noviembre de 2026
Versión	1.0

Contenido

1. Introducción 3
1.1 Propósito y alcance 3
1.2 Estilo arquitectónico 3
1.3 Regla de dependencia y cómo se verifica 3
2. Nivel 1: Contexto del sistema 4
3. Nivel 2: Contenedores 5
4. Nivel 3: Componentes de la API 7
5. Nivel 4 (opcional): Código 9
6. Estructura de carpetas del backend 10
7. Decisiones técnicas justificadas 11
8. Fragmento del módulo de NestJS 12
9. Recorrido de una petición 13
10. Pruebas, CI y limitaciones conocidas 13
Limitaciones conocidas 13

1. Introducción

1.1 Propósito y alcance

Este documento describe la arquitectura de **Guías CLENA**, la página web de autoaprendizaje para la sala de internet del CLENA. El sistema ofrece un catálogo de guías por categoría administrable por el personal, la búsqueda de guías, el registro de solicitudes de ayuda presencial, la autenticación con tres roles y la gestión de usuarios.

La arquitectura se explica con el modelo C4 (contexto, contenedores y componentes, más un nivel de código) y se complementa con la estructura real de carpetas del backend, las decisiones técnicas y un fragmento de código del módulo que une un caso de uso con su repositorio.

1.2 Estilo arquitectónico

El backend sigue **Clean Architecture** con cuatro capas por módulo. Las dependencias apuntan siempre hacia el dominio: el dominio no conoce a nadie, la aplicación solo conoce al dominio, y las capas externas (presentación e infraestructura) dependen de las internas.

Capa	Qué contiene	Carpeta	Puede depender de
Dominio	Entidades, objetos de valor, interfaces de repositorio y puertos.	modules/<módulo>/domain	Solo de primitivos compartidos (paginación, slug).
Aplicación	Casos de uso y errores de aplicación.	modules/<módulo>/application	Dominio y primitivos compartidos.
Infraestructura	Repositorios Prisma, hasher bcrypt y emisor JWT.	modules/<módulo>/infrastructure	Dominio (implementa sus interfaces).
Presentación	Controladores REST, DTOs, guards y decoradores.	modules/<módulo>/presentation	Aplicación y el transporte compartido.

1.3 Regla de dependencia y cómo se verifica

La regla no queda solo en el papel: el archivo `backend/.dependency-cruiser.cjs` define reglas que `dependency-cruiser` comprueba con el comando `npm run architecture:check`. Las reglas más importantes son:

- El **dominio** solo puede importar otros archivos del dominio y primitivos compartidos.
- La **aplicación** solo puede importar dominio, aplicación y primitivos compartidos.
- Ni el dominio ni la aplicación pueden importar NestJS ni Prisma, ni ninguna otra librería externa.
- La **infraestructura** no puede depender de la aplicación ni de la presentación.
- La **presentación** no puede depender de la infraestructura: recibe los casos de uso ya construidos.

Existe además un caso de prueba (`npm run architecture:test-fixture`) que comprueba que un archivo de dominio que importe Prisma sea rechazado. Hoy la verificación se ejecuta de forma manual y no forma parte de los pasos de CI; se recomienda agregarla al flujo de GitHub Actions.

2. Nivel 1: Contexto del sistema

El sistema se muestra como una sola caja, con las personas que lo usan y los sistemas externos con los que se relaciona.

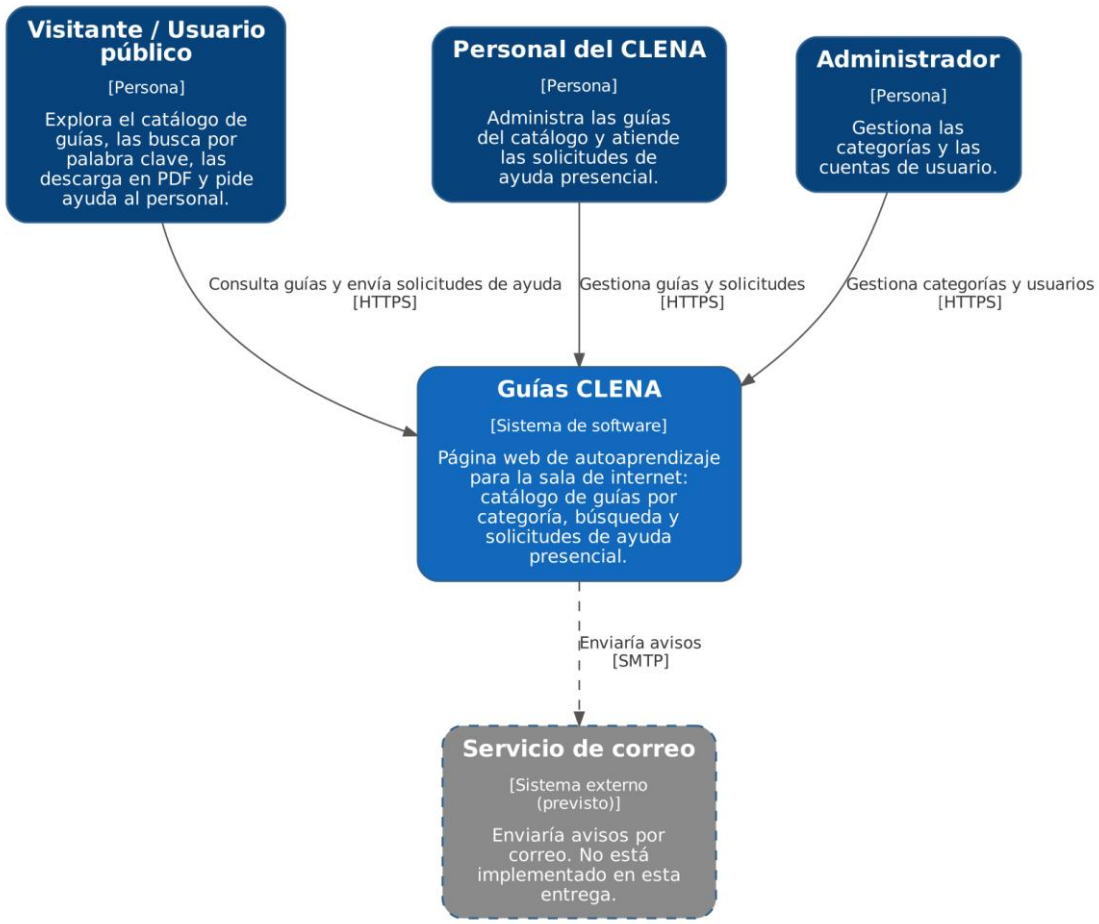


Figura 1. Diagrama C4 de nivel 1: contexto de Guías CLENA.

Persona	Sistema	Contenedor	Componente	Externo (previsto, borde punteado)

Elemento	Tipo	Descripción
Visitante / Usuario público	Persona	Explora el catálogo, busca guías por palabra clave, las descarga en PDF, pide ayuda al personal y consulta sus solicitudes. Rol PUBLIC_USER cuando inicia sesión.
Personal del CLENA	Persona	Rol STAFF. Crea, edita, publica y elimina guías, y atiende las solicitudes de ayuda presencial.
Administrador	Persona	Rol ADMIN. Gestiona las categorías y las cuentas de usuario, además de todo lo que hace el personal.
Guías CLENA	Sistema	Aplicación web con su API y su base de datos.
Servicio de correo	Sistema externo (previsto)	Enviaría avisos por correo mediante SMTP. No está implementado en esta entrega; se dibuja punteado.

3. Nivel 2: Contenedores

Al abrir el sistema se ven los contenedores que se ejecutan y la tecnología de cada uno. Con `docker-compose up` se levantan la base de datos, el backend, el frontend y el proxy inverso.

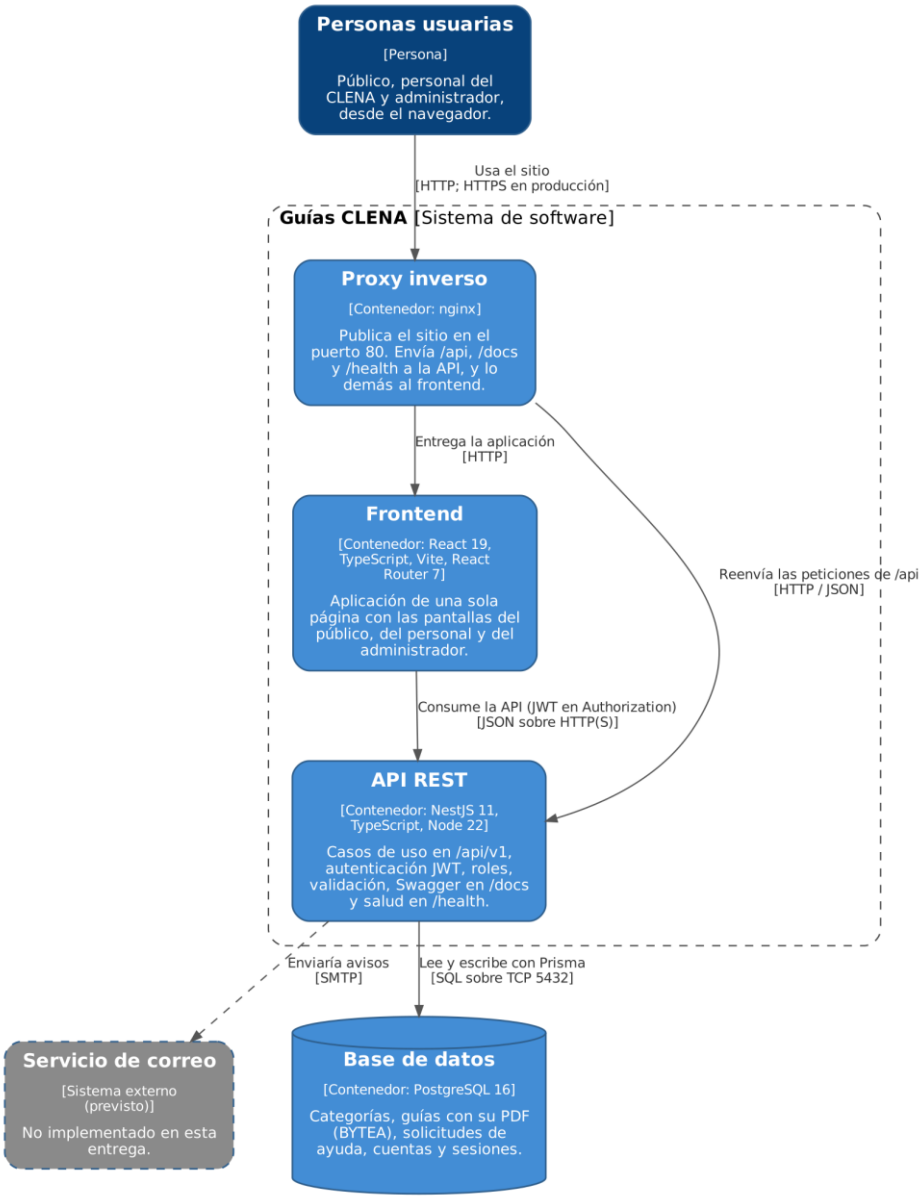


Figura 2. Diagrama C4 de nivel 2: contenedores.

Persona	Sistema	Contenedor	Componente	Externo (previsto, borde punteado)

Contenedor	Tecnología	Responsabilidad	Conexión
Proxy inverso	nginx	Publica el sitio en el puerto 80. Envía `/api`, `/docs` y `/health` a la API, y el resto al frontend. Permite cuerpos de hasta 11 MB, por los PDF.	HTTP (HTTPS en producción)
Frontend	React 19, TypeScript, Vite, React Router 7	Aplicación de una sola página con rutas protegidas según el rol: público, personal y administrador.	HTTP; JSON hacia la API

Contenedor	Tecnología	Responsabilidad	Conexión
API REST	NestJS 11, TypeScript, Node 22	Casos de uso en `/api/v1`, autenticación JWT, roles, validación y Swagger en `/docs`. `/health` queda fuera del prefijo.	HTTP / JSON
Base de datos	PostgreSQL 16 con Prisma 6	Categorías, guías con su PDF (columna BYTEA), solicitudes de ayuda, cuentas y sesiones.	SQL sobre TCP 5432
Servicio de correo (previsto)	SMTP	No implementado en esta entrega.	SMTP

Almacenamiento de archivos. Los PDF de las guías no se guardan en un servicio aparte: viven en la tabla `GuideFile` de PostgreSQL. Por eso este nivel no muestra un contenedor de almacenamiento separado.

Arranque. Al iniciar, el backend ejecuta `prisma migrate deploy` y, en desarrollo con `SEED_DEMO_DATA=true`, carga los datos de demostración. No se necesitan pasos manuales.

4. Nivel 3: Componentes de la API

Este nivel abre el contenedor de la API y muestra sus componentes agrupados en las cuatro capas de Clean Architecture. Las flechas de dependencia apuntan hacia el dominio: la presentación invoca a la aplicación, la aplicación usa el dominio, y la infraestructura implementa las interfaces del dominio.

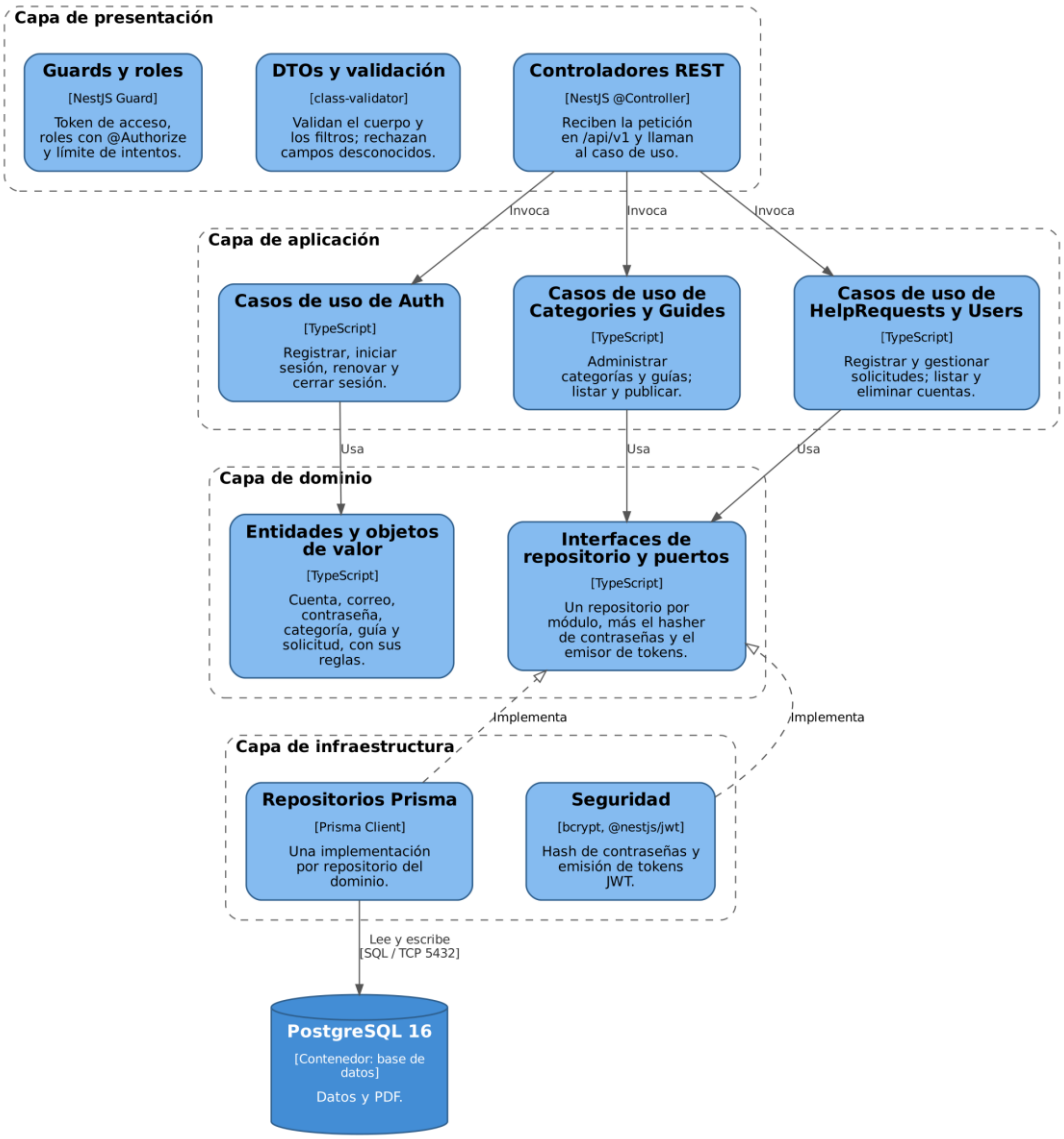


Figura 3. Diagrama C4 de nivel 3: componentes de la API en las cuatro capas.

Contenedor	Componente

Capa	Componente	Elementos reales
Presentación	Controladores	AuthController, CategoriesController, GuidesController, HelpRequestsController, UsersController y HealthController.
Presentación	DTOs y validación	DTOs con class-validator por módulo; `ValidationPipe` global con `whitelist`, `forbidNonWhitelisted` y `transform`.
Presentación	Guards y roles	AccessTokenGuard, RolesGuard con el decorador `@Authorize(...)` y AuthThrottleGuard (límite de intentos).
Presentación	Filtro de excepciones	GlobalExceptionFilter (en `shared/errors`): respuesta uniforme con `statusCode`, `code`, `message`, `timestamp` y `path`.

Capa	Componente	Elementos reales
Aplicación	Casos de uso de Auth	RegisterUseCase, LoginUseCase, RefreshUseCase y LogoutUseCase.
Aplicación	Casos de uso de Categories	CreateCategory, RenameCategory, DeleteCategory y ListCategories.
Aplicación	Casos de uso de Guides	CreateGuide, UpdateGuide, SetGuidePublication, DeleteGuide, ListGuides, ListManagedGuides, GetGuide y GetGuideFile.
Aplicación	Casos de uso de HelpRequests	CreateHelpRequest, ListMyHelpRequests, ListHelpRequests, GetHelpRequest, ChangeHelpRequestStatus y DeleteHelpRequest.
Aplicación	Casos de uso de Users	ListUsers, DeleteUserAccount y ChangeUserStatus.
Dominio	Entidades y objetos de valor	AuthUser, AuthSession, Email, Password, Category, Guide, HelpRequest y UserSummary.
Dominio	Interfaces y puertos	AuthRepository, CategoryRepository, GuideRepository, HelpRequestRepository, UserRepository, UserListRepository, UserAccountRepository, UserStatusRepository, CredentialHasher y TokenIssuer.
Infraestructura	Repositorios Prisma	PrismaAuthRepository, PrismaCategoryRepository, PrismaGuideRepository, PrismaHelpRequestRepository y PrismaUserRepository.
Infraestructura	Seguridad	BcryptCredentialHasher (bcrypt) y JwtTokenIssuer (@nestjs/jwt).
Compartido	Configuración, base de datos y logs	ConfigModule con validación Joi de variables de entorno, DatabaseModule y PrismaService, LoggerModule con nestjs-pino, primitivos de paginación y slug.

5. Nivel 4: Código

Este nivel muestra, para un caso de uso concreto (`CreateCategoryUseCase`), cómo recibe su repositorio por un **token de inyección**. El caso de uso solo conoce la interfaz `CategoryRepository`, que pertenece al dominio. El módulo de NestJS es el único que sabe que esa interfaz se cumple con `PrismaCategoryRepository`.

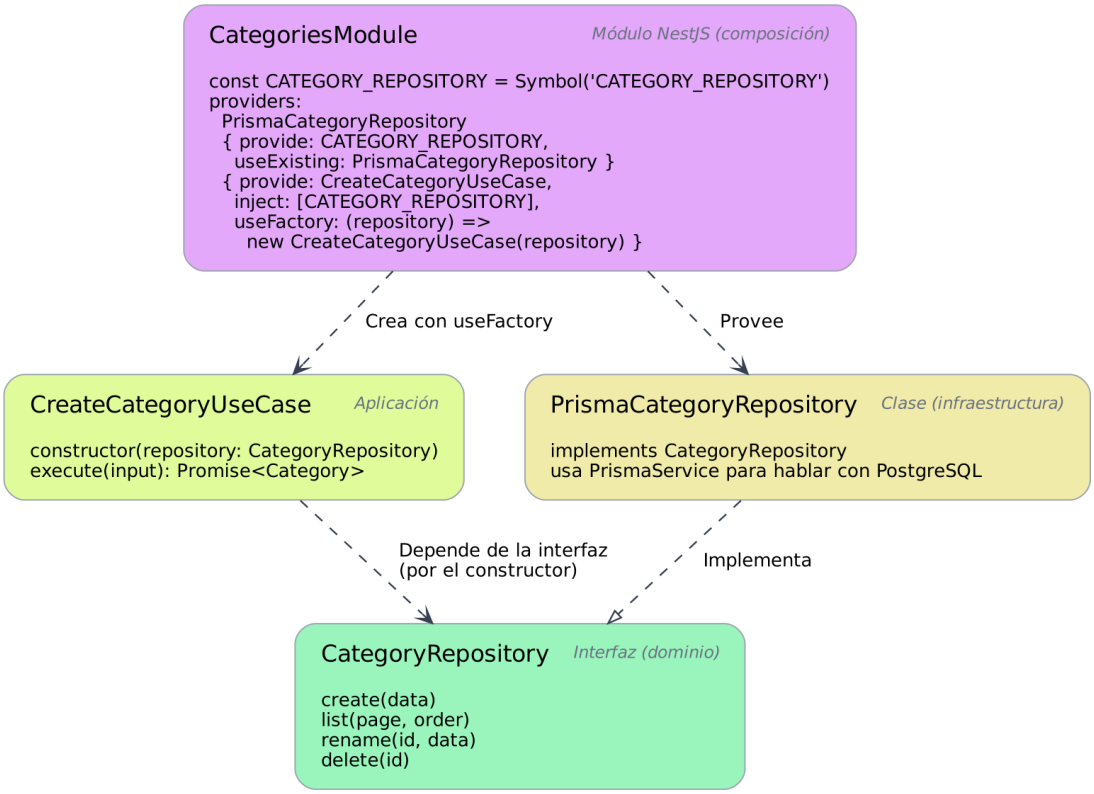


Figura 4. Nivel 4: un caso de uso que recibe su repositorio por token de inyección.

6. Estructura de carpetas del backend

Estructura real de `backend/`. Cada módulo repite las cuatro capas.

```
backend/
├─ prisma/
│   ├─ schema.prisma            modelo de datos
│   ├─ migrations/              migraciones versionadas (sin synchronize)
│   ├─ seed.ts, demo-accounts.ts  datos de demostración
├─ src/
│   ├─ main.ts                  arranque: CORS, prefijo /api/v1, pipes, Swagger
│   ├─ app.module.ts            módulo raíz
│   ├─ modules/
│   │   ├─ auth/
│   │   │   ├─ domain/            entidades, objetos de valor, puertos
│   │   │   ├─ application/       casos de uso y errores
│   │   │   ├─ infrastructure/    repositorio Prisma, bcrypt, JWT
│   │   │   ├─ presentation/      controlador, DTOs, guards
│   │   │   └─ auth.module.ts     composición (inyección por token)
│   │   ├─ categories/   (misma estructura)
│   │   ├─ guides/       (misma estructura)
│   │   ├─ help-requests/ (misma estructura)
│   │   ├─ users/        (misma estructura)
│   │   └─ health/       controlador de salud
│   └─ shared/
│       ├─ config/              validación de variables de entorno (Joi)
│       ├─ database/            PrismaService y errores de Prisma
│       ├─ errors/              errores de aplicación y filtro global
│       ├─ logging/             nestjs-pino con redacción de datos sensibles
│       ├─ primitives/          paginación y slug
│       └─ transport/           DTO de paginación
├─ test/                        e2e, integración, arquitectura
├─ .dependency-cruiser.cjs      reglas de dependencia entre capas
├─ Dockerfile, docker-entrypoint.sh
└─ package.json
```

Tipo de archivo	Dónde va
Entidad, objeto de valor o interfaz de repositorio	`src/modules/<módulo>/domain/`
Caso de uso y su error de aplicación	`src/modules/<módulo>/application/`
Repositorio Prisma, hasher, emisor de tokens	`src/modules/<módulo>/infrastructure/`
Controlador, DTO, guard o decorador	`src/modules/<módulo>/presentation/`
Módulo de NestJS que une casos de uso e implementaciones	`src/modules/<módulo>/<módulo>.module.ts`
Utilidad usada por varios módulos	`src/shared/`
Tabla, migración y semilla	`prisma/`
Prueba unitaria	Junto al archivo probado, con la terminación `.spec.ts`
Prueba e2e, de integración o de arquitectura	`test/`

Frontend (`frontend/`): `src/roles/{public,staff,admin}/pages` contiene las pantallas por rol, `src/auth` la sesión, `src/components` y `src/shared` los elementos comunes, `src/features` funciones como la gestión de contenido y el aviso de privacidad, `services/` las llamadas a la API y `tests/` las pruebas.

7. Decisiones técnicas justificadas

Decisión	Elección	Justificación	Contrapartida
ORM	Prisma 6.19	Esquema declarativo, migraciones versionadas con `prisma migrate deploy` (sin `synchronize`), cliente tipado y transacciones sencillas.	El dominio no puede importar Prisma; por eso cada repositorio traduce filas a entidades.
Validación	class-validator y class-transformer en los DTOs; Joi para las variables de entorno	Los decoradores se integran con NestJS y Swagger. El `ValidationPipe` global rechaza campos desconocidos. Las reglas del dominio (por ejemplo el largo de un título) se repiten en la capa de aplicación para no depender del framework.	Hay dos lugares donde mirar una regla: el DTO y el caso de uso.
Logger	nestjs-pino	Logs JSON estructurados y rápidos. Redacta campos sensibles como contraseñas, cabecera Authorization y tokens.	Los logs en JSON son menos cómodos de leer sin una herramienta.
Frontend	React 19, TypeScript, Vite y React Router 7, sin librería de componentes	Control total del contraste, el tamaño de letra y el foco del teclado, que son requisitos para adultos mayores. El paquete queda pequeño.	Los componentes comunes se escriben a mano.
Autenticación	JWT de acceso (15 min) y de renovación (7 días), con bcrypt	El token de renovación viaja en una cookie HttpOnly y se rota. Hay un límite de intentos en las rutas de autenticación.	Requiere cuidar CORS y credenciales en el cliente.
Roles	ADMIN, STAFF y PUBLIC_USER con `@Authorize`	Permisos declarados en cada ruta, visibles en el controlador y comprobados con pruebas.	Si crecen los roles, convendría una política más fina.
Errores	Filtro global y `ApplicationError`	Todas las respuestas de error tienen la misma forma y la capa de aplicación no depende de HTTP.	Cada error nuevo debe llevar un código estable.
Documentación de la API	Swagger en `/docs`	Documenta los endpoints, los DTOs y el esquema Bearer, y sirve para probar sin frontend.	Hay que mantener los decoradores al día.
Archivos PDF	PostgreSQL, columna BYTEA, máximo 10 MB	Un solo respaldo y un despliegue más simple con `docker-compose`.	La base crece con los PDF. Si el volumen aumenta, conviene un almacenamiento de objetos.
Despliegue	Docker Compose: PostgreSQL 16, backend, frontend y nginx	Una sola orden levanta todo y el backend aplica migraciones y semilla al iniciar.	La configuración de nginx está pensada para desarrollo.
Pruebas y CI	Jest, supertest y GitHub Actions	Pruebas unitarias de casos de uso, e2e de endpoints e integración contra PostgreSQL. El CI ejecuta lint, pruebas y compilación.	Las pruebas de integración necesitan una base de datos.

8. Fragmento del módulo de NestJS

El módulo `CategoriesModule` une el caso de uso con la implementación de su repositorio. Esta es la forma en que la regla de dependencia se cumple en la práctica: el token `CATEGORY_REPOSITORY` apunta a `PrismaCategoryRepository`, y los casos de uso reciben la interfaz.

```
const CATEGORY_REPOSITORY = Symbol('CATEGORY_REPOSITORY');

@Module({
  imports: [AuthModule, DatabaseModule, JwtModule.register({})],
  controllers: [CategoriesController],
  providers: [
    PrismaCategoryRepository,
    { provide: CATEGORY_REPOSITORY, useExisting: PrismaCategoryRepository },
    {
      provide: CreateCategoryUseCase,
      inject: [CATEGORY_REPOSITORY],
      useFactory: (repository: CategoryRepository) =>
        new CreateCategoryUseCase(repository),
    },
    // ListCategoriesUseCase, RenameCategoryUseCase y DeleteCategoryUseCase
    // se registran del mismo modo.
  ],
})
export class CategoriesModule {}
```

El caso de uso, por su parte, no importa nada de NestJS ni de Prisma:

```
export class CreateCategoryUseCase {
  constructor(private readonly repository: CategoryRepository) {}

  async execute(input: CreateCategoryInput): Promise<Category> {
    const result = await this.repository.create(parseCategoryTitle(input.title));
    if (result.kind === 'duplicate') {
      throw new CategoryApplicationError(409, 'CATEGORY_ALREADY_EXISTS', '...');
    }
    return result.category;
  }
}
```

Como el caso de uso depende de una interfaz, sus pruebas unitarias usan un repositorio falso en memoria y no necesitan base de datos.

9. Recorrido de una petición

Ejemplo: el personal despublica una guía con `PATCH /api/v1/guides/:id/publication`.

Paso	Quién	Qué ocurre
1	Navegador y nginx	El frontend envía la petición con el token de acceso en la cabecera `Authorization`. nginx la reenvía a la API.
2	main.ts	Se aplican CORS, el prefijo `/api/v1` y el `ValidationPipe` global.
3	Guards	AccessTokenGuard comprueba el token y RolesGuard exige el rol STAFF o ADMIN.
4	DTO	`SetGuidePublicationDto` valida que `published` sea booleano y rechaza campos desconocidos.
5	Controlador	GuidesController llama a `SetGuidePublicationUseCase`.
6	Caso de uso	Decide el estado destino (Publicada o Borrador) y pide el cambio a la interfaz `GuideRepository`.
7	Repositorio Prisma	Actualiza la fila en PostgreSQL y devuelve la guía.
8	Respuesta	El controlador la convierte en la respuesta JSON. Si algo falla, GlobalExceptionFilter devuelve el error uniforme.

10. Pruebas, CI y limitaciones conocidas

Tipo	Herramienta	Comando
Unitarias (casos de uso, dominio)	Jest	`npm test`
Endpoints de extremo a extremo (e2e)	Jest y supertest	`npm run test:e2e`
Repositorios contra PostgreSQL	Jest	`npm run test:integration`
Reglas de arquitectura	dependency-cruiser	`npm run architecture:check`

El flujo de GitHub Actions se ejecuta en cada Pull Request y corre, en el backend, `npm run lint:check`, `npm test` y `npm run build`, y en el frontend su lint, sus pruebas y su compilación.

Limitaciones conocidas

- **Correo:** el servicio de correo está previsto, pero no implementado; por eso aparece punteado en los diagramas.
- **Verificación de capas en CI:** `npm run architecture:check` no se ejecuta aún en el CI.
- **PDF en la base de datos:** adecuado para el volumen actual; habría que reevaluarlo si crece.